DIGITAL DESIGN PROJECT

Secure UART Communication System

UART Interface with Hardware AES-128 Encryption and CRC-16/XMODEM Error Detection

Prepared by: **Mina Moawad** , **Muhammed Afifi** , **Hoda Ayman**

Program: **Digital Design Phase**

Supervisor: **Eng/Muhammed Salah**

Institution: **NTI**

Date: 7/9/2026

Table of Contents

Right-click the table above and choose "Update Field" (or "Update entire table") after editing this document, so the page numbers and entries reflect your final content.

Document Control

Document Item	Value
Project Scope	UART communication with hardware AES-128 encryption and CRC-16/XMODEM integrity checking
Implementation	Verilog HDL / RTL
Cryptographic Algorithm	AES-128 encryption
Integrity Algorithm	CRC-16/XMODEM
Communication	UART
System Status	Individual modules implemented and integrated into a complete RTL system
Document Type	Technical Design, Architecture, Operation and Verification Report

Revision History

Revision	Date	Description
1.0	September 2026	Initial documentation of the completed UART + AES-128 + CRC-16/XMODEM RTL system

Abstract

This report documents a complete hardware communication and security datapath implemented in Verilog/SystemVerilog HDL. The design combines a UART communication interface, an AES-128 encryption engine, and a CRC-16/XMODEM error-detection engine into an integrated RTL system. Application data is encrypted with AES-128, a CRC-16/XMODEM checksum is calculated over the resulting ciphertext, and the protected frame is transmitted serially through UART. The report covers the project's motivation and requirements, a review of related work, the system architecture, the detailed design of each RTL module (UART, AES-128, CRC-16/XMODEM), the end-to-end integration and handshake strategy, the verification plan (including the AES-128 known-answer vector and the CRC-16/XMODEM standard check value).

1. Project Overview

This project implements a complete hardware communication and security datapath in Verilog/SystemVerilog HDL. The design combines a UART communication interface, an AES-128 encryption engine, and a CRC-16/XMODEM error-detection engine into an integrated RTL system. The objective is to demonstrate how communication, confidentiality, and data-integrity functions can be implemented and coordinated at the hardware level.

The system accepts application data, encrypts the data using AES-128, calculates a CRC-16/XMODEM checksum over the defined protected data field, and transmits the resulting frame through UART. On the receiving side, the corresponding data can be recovered and checked using the matching

protocol. The integrated architecture therefore provides three distinct functions: reliable serial transport, confidentiality, and error detection.

1.1 Project Objectives

- Implement a synthesizable UART transmitter/receiver subsystem in Verilog/SystemVerilog HDL.
- Implement AES-128 encryption as a hardware RTL block.
- Implement CRC-16/XMODEM using the standard CRC parameters and verify its calculation.
- Integrate the three functions into a single end-to-end communication datapath.
- Define clear start/busy/done handshaking between processing stages.
- Verify individual modules and the integrated system using simulation-based testbenches.
- Produce an RTL architecture that can be synthesized and used as a foundation for FPGA or ASIC implementation.

1.2 Functional Summary

Block	Primary Function	Main Data	Completion / Status
UART	Serial communication	Parallel byte ↔ serial bitstream	busy / done and RX indication
AES-128	Confidentiality	128-bit plaintext + 128-bit key ⟶ 128-bit ciphertext	busy / done
CRC-16/XMODEM	Integrity / error detection	Input data ⟶ 16-bit CRC	busy / done
Top-Level Integration	Sequencing and framing	End-to-end protected payload	System control / status

2. System Requirements

2.1 Functional Requirements

- The UART shall serialize and deserialize data according to the selected baud-rate configuration.
- The UART transmitter shall append the required start and stop bits to each transmitted byte.
- The UART receiver shall detect the incoming frame and reconstruct the received byte.
- The AES block shall perform AES-128 encryption on a 128-bit plaintext using a 128-bit master key.
- The CRC block shall calculate a 16-bit CRC using the CRC-16/XMODEM algorithm.
- The integrated controller shall prevent downstream stages from consuming incomplete or invalid data.
- The integrated system shall indicate completion and/or availability of results through defined status signals.

2.2 Assumed / Configurable Parameters

Parameter	Description	Project Treatment
UART clock	System clock used by the UART timing logic	Defined by the implementation/testbench
Baud rate	Serial bit rate	Configurable/selected according to the target system
UART frame	Start + data + stop format	Implemented by the UART subsystem
AES key size	Master key length	128 bits
AES block size	Data block length	128 bits
CRC width	Checksum width	16 bits
CRC polynomial	Generator polynomial	x^16 + x^12 + x^5 + 1 (0x1021)
CRC initial value	Initial register state	0x0000
CRC input reflection	RefIn	False
CRC output reflection	RefOut	False
CRC final XOR	XorOut	0x0000

3. Literature Review

Several published works motivate and inform the design choices made in this project.

3.1 AES Integration with UART

Basu, Kole and Rahaman (2012) describe an early implementation of the AES algorithm inside a UART module to secure serial data transfer, showing that block-cipher encryption can be embedded directly in the serial datapath rather than performed in software on a host processor. Katkade and Phade (2016) extend this idea, applying AES specifically to protect data confidentiality in serial communication links and reporting on the practical trade-offs of adding a cipher to a low-resource serial interface. *[1]*

Kala, Panda and Tanwar (2023) target a high-throughput, low-power AES-128 cipher implementation on FPGA, highlighting the area/throughput trade-off between fully unrolled (pipelined) AES round datapaths and iterative, single-round-reuse datapaths — the latter being the approach adopted for the AES-128 core documented in Section 6, where the same physical round datapath is reused across all ten rounds under FSM control. *[3]*

Kinshuk et al. present a system that encrypts a 128-bit plaintext block with a fixed AES key for secure UART transfer, reinforcing the practice of pairing AES confidentiality with an explicit, well-documented frame convention — the approach followed here with the ciphertext-plus-CRC protected payload defined in Section 4. *[4]*

3.2 Error Detection over Serial Links

Because UART itself provides no automatic-repeat-request or acknowledgment mechanism, corrupted or truncated frames can otherwise be silently accepted by a receiver. Prior work combining UART with CRC-based error detection demonstrates that a cyclic redundancy check substantially improves reliability compared with parity alone, particularly for multi-byte frames where several bits may be corrupted by burst noise. This project uses the CRC-16/XMODEM parameterization specifically — polynomial 0x1021 with a non-reflected, zero-initialized register (init 0x0000, RefIn/RefOut false, XorOut 0x0000) — which differs from the closely related CRC-16/CCITT-FALSE variant only in its initial value.

3.3 Positioning of This Project

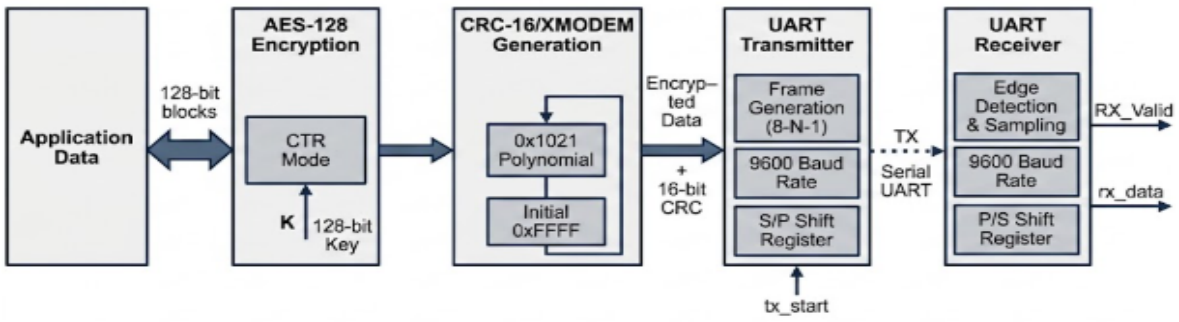
Relative to the works above, this project's contribution is the coordination of three independently verifiable RTL blocks — an iterative AES-128 core, a byte-oriented CRC-16/XMODEM engine, and a UART transceiver — through an explicit start/busy/done handshake protocol and a top-level FSM, together with a documented verification plan built on standard, published test vectors for both AES-128 and CRC-16/XMODEM.

4. System Architecture

The integrated system is organized as a sequence of hardware processing stages. The exact top-level framing can be adapted to the selected application, but the logical relationship between the implemented functions is fixed.

4.1 Transmit path

Application Data → AES-128 Encryption → CRC-16/XMODEM Generation → UART Transmitter → UART RX



The AES-128 Decryption stage above is not yet implemented in the delivered RTL — Section 6 documents the encryption core only, and a matching decryption path is listed as future work in Section 12. It is included in this table to show where it will fit once built, and should be treated as planned rather than current until an aes128_decrypt module (or equivalent inverse-round datapath) is added alongside Section 6.

4.2 Block-Level Responsibilities

Stage	Input	Processing	Output
Input / Application	Raw application data	Load and validate payload	128-bit data block / byte stream
AES-128	128-bit plaintext + 128-bit key	Key expansion + 10 encryption rounds	128-bit ciphertext
CRC-16/XMODEM	Defined protected data field	Polynomial division / CRC update	16-bit CRC
Framing	Ciphertext + CRC	Construct transmission sequence	UART byte stream
UART TX	Parallel byte `data[7:0]` + `tx_en` request	Frame assembly `{stop,data[7:0],start}` (LSB-first) and tick-driven bit-serial shift-out via the IDLE/ON FSM	Serial TX bitstream (`tx`), plus `busy`/`done` status
UART RX	Serial RX bitstream	Falling-edge detection → 1.5-bit then 1-bit mid-bit sampling via the down-counter → SIPO shift-in → stop-bit validation	Reassembled byte `data[7:0]`, plus `busy`/`done`/`err` status
AES-128 Decryption	128-bit Ciphertext + 128-bit key	Key expansion + 10 Decryption rounds	128-bit plaintext
CRC Check	Data with CRC bits	Polynomial division/check CRC	Valid or corrupted frame

4.3 Protected Frame Format

A practical protected frame is organized as ciphertext followed by CRC. The exact byte ordering is fixed between transmitter and receiver and documented as part of the interface specification.

Field	Size	Purpose
Ciphertext	16 bytes	AES-128 encrypted application block
CRC	2 bytes	CRC-16/XMODEM integrity check
Total protected payload	18 bytes	Data transmitted after framing into UART bytes

Important framing rule

The CRC must be calculated over exactly the same byte sequence that the receiver uses for verification. The specification should explicitly define whether the CRC covers plaintext, ciphertext, header fields, or another protected field.

4.4 Byte Serialization of the AES Result

The 128-bit AES result is converted into 16 bytes for UART transmission. The implementation must define a consistent byte ordering.The most-significant byte is transmitted first, the sequence is Ciphertext[127:120] → Byte 0, Ciphertext[119:112] → Byte 1, …, Ciphertext[7:0] → Byte 15, followed by the two CRC bytes in the documented order (high byte then low byte).

5. UART Communication Module

UART (Universal Asynchronous Receiver/Transmitter) provides the serial communication interface of the system. Because UART is asynchronous at the physical protocol level, the transmitter generates the serial waveform from the local clock while the receiver reconstructs data from the incoming RX signal using its configured bit timing. The system operates on a standard 8-N-1 frame format (1 start bit, 8 data bits LSB first, no parity bit, 1 stop bit) at a default baud rate of 9600 baud using a 50 MHz system clock.

5.1 UART Transmitter Operation

1. **Idle State:** Wait in the idle state (`IDLE`) with the `tx` output line held high (logic 1).
2. **Load Frame:** Construct 10-bit frame `{1'b1,data,1'b0}` with start (0), 8 data bits, and stop (1) bits
3. **Transmit:** Transition to `ON` state, assert `busy`, and output bits sequentially indexed by baud rate `tick` pulses
4. **Complete:** Clear `busy`, pulse `done` high, reset index, and return to `IDLE` after transmitting all 10 bits.

The `busy` signal is important at the system level: it prevents a new byte from overwriting the currently transmitted byte. The `done` signal can be used by the integrated controller to advance to the next byte. .

5.2 UART Receiver Operation

1. **Start Detection:** Monitor `rx` in `IDLE`; assert `falling_edge` on a high-to-low transition (start bit).
2. **Mid-Bit Sampling:** Trigger the baud counter to wait 1.5 x bit period (7812 cycles) to Sample the start bit midpoint, then reload 1 x bit period (5208 cycles) for subsequent Data bits
3. **Data Shift:** Assert `en_shift` on each `finish` tick to shift incoming `rx` bits LSB-first into the SIPO register (`{rx, data[7:1]}`).
4. **Stop Bit & Validation:** Sample the stop bit after 8 data bits; assert `done` if `rx` is high (logic1), or enter `ERR` and assert `err` if low (logic 0).

5.3 UART Interface

Signal	Direction	Description
clk	Input	System clock signal (50 MHz)
rst	Input	synchronous active-high reset signal
arst_n	Input	Asynchronous active-low reset signal
tx_en	Input	Enable signal to initiate transmission of loaded byte
`data` (TX)	Input	8-bit parallel byte to transmit
tx	Output	Serial UART transmit line
`busy` (TX)	Output	High while a byte transmission is in progress
`done` (TX)	Output	Pulse high indicating completion of byte transmission
`rx_en`	Input	Enable signal for receiver module
rx	Input	Serial UART receive line
`data` (RX)	Output	8-bit recovered received data byte
`done` (RX)	Output	Pulse high indicating valid byte successfully received
`busy` (RX)	Output	High while frame reception is in progress
`err`	Output	indicates a framing error (invalid stop bit)

Integration note

The UART interface is byte-oriented, while AES operates on 128-bit blocks and CRC produces a 16-bit result. The top-level controller is therefore responsible for converting the block-level results into an ordered byte stream.

6. AES-128 Encryption Module

AES (Advanced Encryption Standard) is used to provide confidentiality. The implemented AES engine operates on a 128-bit plaintext block and a 128-bit master key, producing a 128-bit ciphertext block. AES-128 uses 10 encryption rounds in addition to the initial AddRoundKey operation.

6.1 AES-128 Encryption Structure

Operation	Purpose
Initial AddRoundKey	XOR the plaintext state with the initial round key.
Rounds 1–9	SubBytes → ShiftRows → MixColumns → AddRoundKey.
Round 10	SubBytes → ShiftRows → AddRoundKey; MixColumns is omitted.

6.2 AES State Transformation

- SubBytes: substitutes each byte using the AES S-box.
- ShiftRows: cyclically shifts rows of the 4×4 byte state.
- MixColumns: applies the AES matrix transformation to each column.
- AddRoundKey: XORs the state with the current 128-bit round key.

6.3 AES Key Expansion

The AES-128 key schedule expands the 128-bit master key into eleven 128-bit round keys, K0 through K10. K0 is used by the initial AddRoundKey operation, while K1 through K10 correspond to encryption rounds 1 through 10.

```
AES-128 key schedule

Master Key -> K0
  |
  K1 -> Round 1
  K2 -> Round 2
  ...
  K9 -> Round 9
  K10 -> Round 10
```

For encryption, the round-key order is therefore K0 at the initial transformation followed by K1, K2, …, K10. For a corresponding AES decryption implementation (noted as future work in Section 11), the round keys are consumed in reverse order, subject to the exact inverse-round architecture chosen.

6.4 AES Control Sequence

State	Action
IDLE	Wait for a new encryption request.
LOAD	Capture plaintext and master key.

State	Action
KEY_EXPANSION	Generate/store the AES round keys.
INITIAL_ADD_KEY	Perform plaintext XOR K0.
ROUND_1 – ROUND_9	Execute the standard full AES round.
ROUND_10	Execute the final AES round without MixColumns.
DONE	Register the ciphertext and signal completion.

A clocked FSM-based implementation is suitable for this architecture because it makes the sequencing of the ten AES rounds explicit and permits the same physical round datapath to be reused across rounds, minimizing hardware area compared with a fully unrolled implementation (see Section 9.1).

6.5 AES Interface

Signal	Description
clk	System clock
rst_n	Active-low reset, according to the implemented interface
start	Starts an AES encryption operation
plaintext[127:0]	128-bit input data block
master_key[127:0]	128-bit AES-128 master key
ciphertext[127:0]	128-bit encrypted output block
busy	Indicates that AES is processing
done	Indicates that ciphertext is valid

7. CRC-16/XMODEM Module

The CRC module implements a **CRC-16/XMODEM** error-detection algorithm in synthesizable Verilog HDL. Its purpose within the complete UART/AES/CRC system is to provide **error detection for transmitted data**. While AES-128 provides confidentiality, CRC provides a mechanism for detecting accidental corruption of the transmitted information.The implemented module accepts a **128-bit input data word** and generates a **16-bit CRC value**, resulting in a **144-bit output** containing the original data followed by its calculated CRC. The module is parameterized to allow the data width and CRC width to be modified.

7.1 CRC-16/XMODEM Parameters

Parameter	Value
CRC width	16 bits
Polynomial	0x1021
Polynomial expression	x^16 + x^12 + x^5 + 1
Initial value	0x0000
RefIn	False
RefOut	False
XorOut	0x0000
Check value	0x31C3 for the standard ASCII string “123456789”

7.2 CRC Calculation Concept

The CRC engine treats the input as a sequence of bits/bytes and updates a 16-bit register using polynomial division. For a non-reflected CRC-16/XMODEM implementation, each input byte is processed from the most significant bit toward the least significant bit.

```
CRC-16/XMODEM
Polynomial = 16'h1021
Init       = 16'h0000
RefIn      = false
RefOut     = false
XorOut     = 16'h0000

For each input byte:
    CRC <- CRC XOR (byte << 8)
    repeat 8 times:
        if CRC[15] == 1:
            CRC <- (CRC << 1) XOR 16'h1021
        else:
            CRC <- CRC << 1
```

7.3 CRC Interface

Signal	Description
clk	System clock
rst_n	Reset input according to the implementation
data_in	Input data block
busy	High while the CRC calculation is active
done	High/pulsed when the CRC result is available
data_with_crc	Calculated CRC-16/XMODEM value

7.4 CRC Processing Architecture

The CRC generator employs a **sequential bit-by-bit architecture** to calculate the CRC-16/XMODEM checksum over a 128-bit input data block.

The input data is first captured in an internal register, while a 16-bit CRC register stores the intermediate checksum and a counter tracks the number of processed bits. The CRC calculation is initialized to **0x0000** and performed **MSB-first**, starting from `data_in[127]` and progressing to `data_in[0]`. One input bit is processed per clock cycle. For each bit, the feedback value is obtained by XORing the current MSB of the CRC register with the input bit. The CRC register is then shifted left, and the generator polynomial **0x1021** is XORed with the result when the feedback value is logic `1`.

The operation is implemented within a **single clocked process without a dedicated FSM**. When the module is idle (`busy = 0`), the input is captured and the CRC calculation is initialized. While `busy = 1`, the module processes one bit per clock cycle until all 128 bits have been evaluated, requiring **128 processing cycles**. Upon completion, the calculated 16-bit CRC is appended to the original 128-bit data to form a **144-bit output frame**, with the data occupying the upper 128 bits and the CRC occupying the lower 16 bits. The `done` signal indicates completion and the availability of the valid output, while `busy` indicates that the CRC engine is actively processing a data block.

7.5 CRC Testbench

The provided testbench verifies the RTL using three independent 128-bit input vectors.

The testbench first applies reset:

```
data_in = 128'b0;
rst_n  = 1'b0;

#20;
rst_n = 1'b1;
```

After reset is released, each test waits until the CRC module is idle before applying a new input. This is important because the module automatically begins a new calculation when `busy` is low.

The testbench then waits for the `done` event:

```
@(posedge done);
```

Once `done` is asserted, the testbench compares: `data_with_crc[15:0 ]` against the expected CRC value.

The three implemented test cases are:

Test	Input Data	Expected CRC
TEST 1	`31323334353637383941424344454630`	`16'h2952`

TEST 2	00112233445566778899AABBCCDDEEFF	16'h1248
TEST 3	1122334455667788 99AABBCCDDEEAA55	16'h8169

8. End-to-End Data Path and Integration

The integrated system connects the three implemented processing functions through a top-level control layer. The central integration problem is not the individual algorithms but the movement of data between modules with different granularities: AES processes 128-bit blocks, CRC produces a 16-bit checksum, and UART transmits individual serial frames containing one byte at a time.

8.1 Recommended Transmission Sequence

5. Load the application plaintext and AES master key.
6. Start AES encryption.
7. Wait until AES asserts done and the ciphertext is valid.
8. Provide the protected data field to the CRC engine.
9. Start the CRC calculation.
10. Wait until CRC asserts done and crc_out is valid.
11. Build the transmission sequence from ciphertext bytes and CRC bytes.
12. Transmit each byte through UART only when UART busy is low.
13. Wait for UART done before advancing to the next byte.
14. After the final byte, return the integrated controller to its idle state.

8.2 Integrated State Machine

State	Action	Exit Condition
IDLE	Wait for system start/request	Start asserted
AES_START	Launch AES	AES busy
AES_WAIT	Wait for ciphertext	AES done
CRC_START	Launch CRC	CRC busy
CRC_WAIT	Wait for checksum	CRC done
UART_LOAD	Select next byte	UART not busy
UART_SEND	Request transmission	UART busy
UART_WAIT	Wait for byte completion	UART done
COMPLETE	Report complete frame	Return to IDLE

9. Control and Handshake Strategy

A clean handshake protocol is essential because each processing block requires multiple clock cycles. The top-level controller should never assume that an output is available merely because an operation was started.

Signal Pattern	Meaning	Integration Rule
start	One operation requested	Assert only when the target block is ready/idle
busy = 1	Block is processing	Do not overwrite its active input/result registers
done = 1	Operation completed	Capture/use the result and advance the controller
valid	Output data is valid	Consume only when asserted
reset	Return to known state	Clear active operations and status flags

9.1 Busy Protection

The busy signal is especially important for UART transmission. A byte must not be loaded into the UART transmitter while a previous byte is still being serialized. The same principle applies to AES and CRC: their input registers must not be changed during an active operation unless the module explicitly supports pipelining.

9.2 Done Pulse Handling

If done is implemented as a one-clock pulse, the controller should sample it synchronously and transition immediately to the next state. If done remains asserted until acknowledged, the controller must include an acknowledgment or state transition that prevents repeated consumption of the same result.

10. Timing and Synthesizability Considerations

The system is clocked RTL and is intended to be synthesizable. Because AES and CRC contain iterative logic and UART contains baud-rate timing, the main timing considerations are combinational depth, register placement, and the number of clock cycles allocated to each operation.

10.1 AES Timing

- An iterative AES architecture reduces hardware area by reusing the round datapath.
- The critical path can be influenced by the S-box implementation, MixColumns logic, and key-expansion logic.
- Registering intermediate results between rounds can improve timing at the cost of additional latency.
- A fully combinational 10-round implementation may reduce cycle latency but typically increases area and timing pressure.

10.2 CRC Timing

- A bit-serial CRC implementation uses very little hardware but requires multiple cycles per byte.
- A byte-parallel implementation can process one byte per cycle but uses more combinational logic.
- The selected implementation is matched to the required throughput of the UART link.

10.3 UART Timing

- UART baud timing must be derived consistently from the system clock.
- The transmitter must maintain the configured bit period with sufficient accuracy.
- The receiver sample near the center of each bit period for robust operation.

10.4 Reset Strategy

Reset behavior initialize all state machines, counters, shift registers, busy flags, done flags, and data-valid indicators. After reset, the integrated system return to a defined idle state without starting an unintended encryption, CRC calculation, or UART transmission.

11. Design Decisions and Engineering Rationale

Decision	Rationale
Modular RTL	Allows AES, CRC, and UART to be verified independently and reused in other systems.
AES-128	Provides a standardized 128-bit block cipher with a compact key size suitable for an RTL learning/project implementation.
CRC-16/XMODEM	Provides a standardized 16-bit checksum with well-defined parameters and a known validation vector.
Handshake-based control	Prevents data overwrite and makes multi-cycle processing safe.
Byte-oriented UART interface	Matches the natural granularity of UART while allowing the top level to serialize AES/CRC results.
Iterative processing	Offers a practical balance between hardware resource usage and latency for a project-scale RTL implementation.

11.1 Security vs. Integrity

AES and CRC serve different purposes and should not be treated as interchangeable. AES provides confidentiality when used with a properly managed secret key, while CRC detects accidental transmission errors. CRC is not a cryptographic authentication mechanism and should not be used to establish authenticity or protect against an intentional attacker modifying the message.

11.2 Why CRC Is Useful **After** Encryption

A communication link can corrupt ciphertext bits just as it can corrupt plaintext bits. Computing CRC over the defined transmitted protected data allows the receiver to detect many accidental bit errors before accepting the data as valid.

12. Limitations and Future Improvements

- Add a complete AES decryption path for bidirectional secure communication.
- Add a formal frame format with synchronization, length, sequence number, and explicit CRC coverage.
- Add CRC error reporting at the receiver and a defined retransmission policy.
- Optimize the S-box implementation for the selected FPGA/ASIC technology.
- Add assertions and/or formal verification for key control properties such as busy exclusivity and state transitions.
- Add randomized constrained verification and functional coverage.
- Perform post-synthesis timing analysis and document area, power, and maximum-frequency results.

13. Final Specifications

Parameter	Selected Target
AES	AES-128, 128-bit key, 128-bit block, 10 rounds
AES mode	Direct block encryption (no chaining mode)
CRC	CRC-16/XMODEM, polynomial 0x1021, initial value 0x0000, RefIn/RefOut false, XorOut 0x0000
Protected payload	18 bytes (16-byte ciphertext + 2-byte CRC)
UART	Single lane, start/data/stop framing, configurable baud rate
Control style	Synchronous start/busy/done handshaking between all stages
System status	Individual modules implemented and integrated into a complete RTL system

14. Conclusion

The completed project demonstrates the integration of three important digital hardware functions into a single RTL communication system. UART provides the physical serial data transport, AES-128 provides encryption, and CRC-16/XMODEM provides error detection. The resulting architecture illustrates how independently verified RTL blocks can be coordinated through control states and handshaking to form a practical hardware subsystem.

> The project successfully brings UART, AES-128 encryption, and CRC-16/XMODEM into one integrated Verilog RTL system. The architecture is modular and provides a solid basis for FPGA deployment, further verification, and eventual ASIC-oriented refinement.

References

[1] Basu, D., Kole, D. K., Rahaman, H.: “Implementation of AES algorithm in UART module for secured data transfer.” International Conference on Advances in Computing and Communications (2012), pp. 142–145.

[2] Katkade, P., Phade, G.: “Application of AES algorithm for data security in serial communication.” International Conference on Inventive Computation Technologies (2016), pp. 1–5.

[3] Kala, J., Panda, J., Tanwar, L.: “FPGA implementation of a high throughput Low Power Advanced Encryption Standard (AES-128) Cipher.” 10th International Conference on Signal Processing and Integrated Networks (SPIN) (2023), pp. 367–372.

[4] Kinshuk et al.: “Promoting Sustainable Security by Implementing AES Algorithm for Enhanced Confidentiality in UART Serial Communication.” Springer Nature Link.

[5] National Institute of Standards and Technology (NIST): “Advanced Encryption Standard (AES), FIPS PUB 197.” U.S. Department of Commerce.

[6] CRC-16/XMODEM parameterization: polynomial 0x1021, init 0x0000, RefIn/RefOut false, XorOut 0x0000, check value 0x31C3, as catalogued in standard CRC parameter registries.